

# The Production Environment Takeover Handbook

Infrastructure discovery, access auditing,  
undocumented systems, dependency mapping,  
operational ownership, risk assessment,  
and stabilization.



Discover



Audit Access



Identify  
Undocumented  
Systems



Map Dependencies



Establish Ownership



Assess Risks



Stabilize



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# You Have Inherited a Production Environment. What Now?

Your responsibilities, immediate priorities, and why you should not start changing infrastructure.

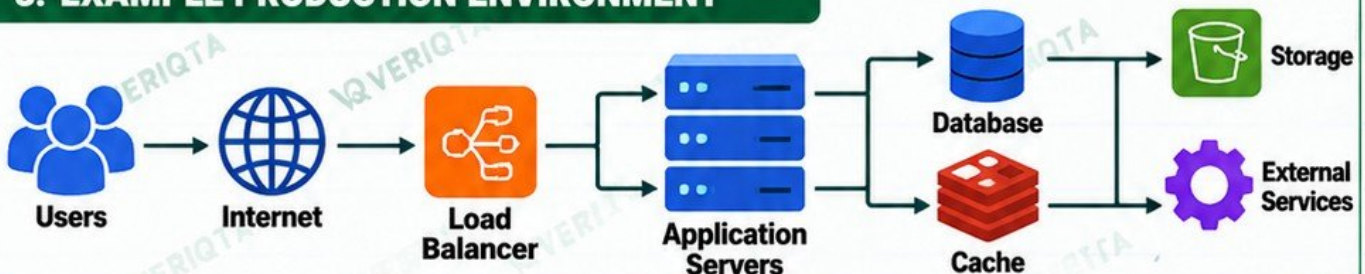
## 1. WHAT DOES THIS MEAN?

- You are now responsible for an existing production environment.
- This environment is already running real applications.
- People, customers, or businesses depend on it.
- Your job is to understand it, keep it stable, and improve it safely.

## 2. WHY YOU SHOULD NOT START CHANGING THINGS

- You do not yet know how everything is connected.
- You can break something you cannot see.
- There may be undocumented dependencies.
- A small change can cause a production outage.
- Your first goal is understanding, not implementation.

## 3. EXAMPLE PRODUCTION ENVIRONMENT



## 4. YOUR KEY RESPONSIBILITIES

- Understand the current infrastructure.
- Keep the environment stable and available.
- Learn how applications and services work.
- Identify risks and gaps.
- Document what you discover.
- Work with the right people.
- Plan improvements with proper approvals.

## 5. IMMEDIATE PRIORITIES

- 1 Get access to read and observe.
- 2 Identify key people and contacts.
- 3 Understand what is running and how it is connected.
- 4 Review monitoring, alerts, and recent incidents.
- 5 Identify critical systems and dependencies.
- 6 Document everything before making changes.



### JUNIOR ENGINEER TIP

In a new production environment, your biggest value is to understand and document, not to make quick changes. A stable system is more important than a fast fix.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# The First 24 Hours in an Unfamiliar Production Environment

What to request, who to contact, what to observe, and what to document.

## 1. YOUR GOAL IN THE FIRST 24 HOURS

- Understand what you have inherited.
- Keep the environment stable.
- Learn how everything is connected.
- Identify immediate risks.
- Collect information before making changes.
- Build a clear action plan.

## 2. WHAT NOT TO DO

- Do not make infrastructure changes.
- Do not restart or stop services.
- Do not update configurations.
- Do not deploy new code.
- Do not delete or clean up anything.
- Do not assume you understand how things work.
- Do not work alone.

## 3. WHO TO CONTACT

-  **Your manager or direct lead**  
Understand expectations and scope.
-  **Existing operations or DevOps team**  
Get environment overview and access.
-  **Application owners**  
Understand business-critical services.
-  **Security team**  
Understand access policies and restrictions.
-  **On-call / incident response team**  
Know who to contact if something breaks.

## 4. WHAT TO REQUEST

-  **Access to required systems**  
Servers, cloud consoles, repositories, monitoring, etc.
-  **Existing documentation**  
Architecture diagrams, runbooks, SOPs.
-  **List of key people and contacts**  
Service owners, on-call, vendors.
-  **Access to monitoring and logs**  
Dashboards, alerting, log platforms.
-  **CI/CD and deployment information**  
Repositories, pipelines, approval process.
-  **Inventory of current infrastructure**  
Servers, databases, containers, cloud resources.

## 5. WHAT TO OBSERVE

-  **Check system health**  
Look at monitoring dashboards and alerts.
-  **Understand what is running**  
Applications, services, databases, containers.
-  **Review recent activity**  
Recent deployments, changes, incidents.
-  **Look for obvious red flags**  
High alerts, failed services, unusual resource usage.
-  **Understand dependencies**  
Which systems rely on each other.

## 6. WHAT TO DOCUMENT

-  **Systems and services you discover.**
-  **Access details and permissions granted.**
-  **Application dependencies.**
-  **Existing configurations and versions.**
-  **Open risks, warnings, or gaps.**
-  **Questions and things you do not understand.**

## 7. END OF DAY 1 CHECKLIST

-  Access obtained to key systems
-  Key contacts identified
-  Initial environment understanding complete
-  List of risks and gaps started
-  Plan for next steps ready



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Discovering Infrastructure Nobody Properly Documented

Identify servers, cloud resources, databases, clusters, networking, and external services.

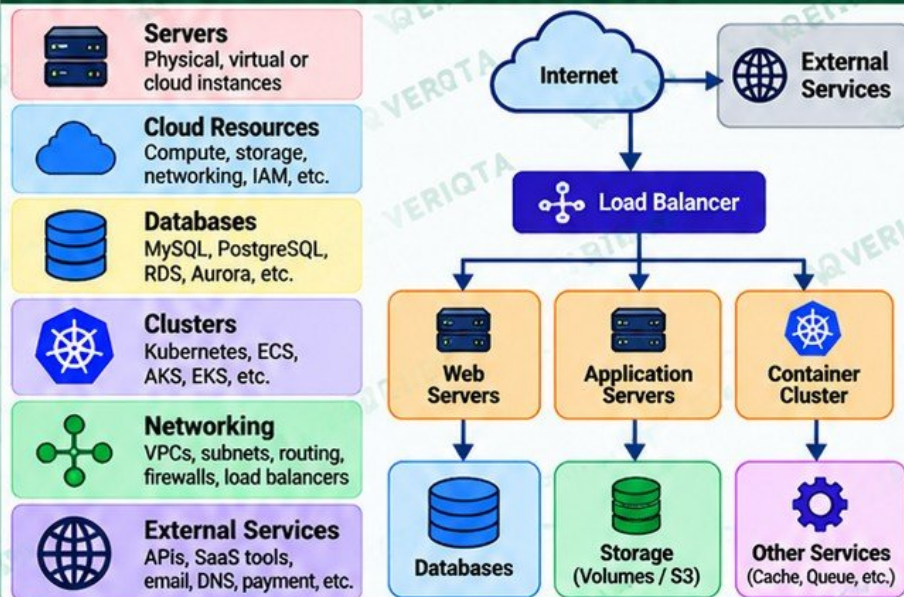
## 1. WHAT IS INFRASTRUCTURE DISCOVERY?

- Finding and listing all the systems that make up the production environment.
- Helps you understand what exists, how it is connected, and what it depends on.
- You look for servers, cloud resources, databases, clusters, networking components, and external services.

## 2. WHY IT IS IMPORTANT

- Many environments have undocumented resources.
- There may be old servers, manual scripts, or forgotten services.
- You cannot secure, monitor, or manage what you do not know exists.
- It helps you identify risks, dependencies, and potential single points of failure.

## 3. COMMON INFRASTRUCTURE COMPONENTS



## 4. HOW TO DISCOVER

- 1 **Check cloud accounts**  
List all regions and resources.
- 2 **Review access and credentials**  
Look for other accounts or tools.
- 3 **Identify running servers**  
Physical, virtual, or cloud.
- 4 **Find databases**  
Managed and self-hosted.
- 5 **Discover container platforms**  
Kubernetes, ECS, etc.
- 6 **Review networking components**  
VPCs, subnets, firewalls, load balancers.
- 7 **Look for external dependencies**  
APIs, SaaS tools, third-party services.
- 8 **Talk to people**  
Teams often know about hidden systems.

## 5. USEFUL COMIMAEADS AND TOOLS

|                                                         |                                     |
|---------------------------------------------------------|-------------------------------------|
| <code>aws resourcegroupstaggingapi get-resources</code> | List AWS resources across accounts. |
| <code>aws ec2 describe-instances</code>                 | List EC2 instances.                 |
| <code>aws rds describe-db-instances</code>              | List RDS databases.                 |
| <code>kubectl get all -A</code>                         | List all Kubernetes resources.      |
| <code>docker ps -a</code>                               | List Docker containers.             |
| <code>systemctl list-units --type=service</code>        | List running services (Linux).      |
| <code>netstat -tulnp   grep LISTEN</code>               | Check open ports.                   |

## 6. DOCUMENT WHAT YOU FIND

- ✓ Resource name and type
- ✓ Environment (prod, staging, etc.)
- ✓ Location (region, network, etc.)
- ✓ Purpose and owner (if known)
- ✓ Dependencies
- ✓ Access method
- ✓ Configuration or version (if available)
- ✓ Observations and gaps



### JUNIOR ENGINEER TIP

Use multiple sources (cloud console, CLI, configuration files and people) because no single tool will show you everything.



### COMMON MISTAKE

Assuming the environment is fully documented. There are usually hidden resources and old systems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Finding Out What Is Actually Running

Inspect active processes, systemd services, containers, workloads, and scheduled jobs.

## 1. CHECK RUNNING PROCESSES

Shows all active processes on the server.

```
ps aux --sort=-%cpu | head -n 10
```

- Lists processes using the most CPU.
- Helps identify what applications are running.
- Useful for spotting unexpected or unknown processes.

## 2. CHECK SYSTEMD SERVICES

Shows running and enabled services.

```
sudo systemctl list-units --type=service --state=running
sudo systemctl list-unit-files --type=service
```

- First command shows active services.
- Second command shows what is enabled to start on boot.
- Helps you understand core system services.



## 3. CHECK CONTAINERS (DOCKER)

Shows running containers on the server.

```
docker ps
```



- Lists all running containers.
- Use `docker ps -a` to see stopped containers.
- Helps identify applications running in containers.

## 4. CHECK KUBERNETES WORKLOADS

Shows running workloads in a cluster.

```
kubectl get pods -A
kubectl get deployments -A
kubectl get services -A
```



- Lists all pods, deployments, and services across all namespaces.
- Helps you understand what applications are running in the cluster.

## 5. CHECK SCHEDULED JOBS

Shows cron jobs for the current user and system.

```
crontab -l
sudo crontab -l
ls -l /etc/cron.d/
```



- Shows scheduled tasks (cron jobs).
- Helps you find automation, backups, and maintenance jobs.
- Look for scripts you were not aware of.

## 6. CHECK LISTENING PORTS

Shows which applications are listening for connections.

```
sudo ss -tlnp
sudo netstat -tlnp
```



- Shows open ports and the processes using them.
- Helps you identify running applications and unexpected services.

## 7. CHECK RESOURCE USAGE

Understand how much CPU, memory, and disk is used.

```
top htop df -h free -h
```

- `top` or `htop` shows real-time resource usage.
- `df -h` shows disk usage.
- `free -h` shows memory usage.



## 8. WHAT TO LOOK FOR



- ✓ Unknown or unexpected processes.
- ✓ Services you do not recognize.
- ✓ Containers you were not told about.
- ✓ Suspicious scheduled jobs.
- ✓ Applications listening on unexpected ports.
- ✓ High CPU, memory, or disk usage.
- ✓ Workloads running in the wrong environment.



### COMMON MISTAKE

Only checking one tool. Always check processes, services, containers, cron jobs, and ports together.



### JUNIOR ENGINEER TIP

Take notes as you explore. Save command output to a file so you can compare later and track changes.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Who Has Access to Production?

Audit users, SSH access, IAM identities, service accounts, privileged access, and stale credentials.

## 1. TYPES OF PRODUCTION ACCESS



### User Accounts

Linux users, cloud users, application users.



### SSH Access

Keys, passwords, bastion hosts.



### Cloud IAM Identities

AWS, Azure, GCP users, roles, policies.



### Service Accounts

Used by applications and automation.



### Privileged Access

Root, sudo, admin roles.



### Stale Credentials

Old users, unused keys, inactive accounts.

## 2. CHECK LINUX USER ACCOUNTS

```
cat /etc/passwd
cat /etc/group
sudo last -a | head -n 20
sudo who
sudo w
```



- Lists local users and groups.
- Shows recent logins.
- Helps identify unexpected or inactive users.
- Look for users you do not recognize.

## 3. CHECK SSH ACCESS

```
cat /etc/ssh/sshd_config
ls -la ~/.ssh/
ls -la /root/.ssh/
sudo cat /etc/ssh/authorized_keys
```



- Check if password login is allowed.
- Review SSH keys and who has access.
- Look for keys that are no longer in use.
- Ensure only authorized users have access.

## 4. CHECK CLOUD IAM IDENTITIES (AWS EXAMPLE)

```
aws iam list-users
aws iam list-roles
aws iam list-access-keys
aws iam get-account-summary
```



- Lists IAM users and roles.
- Checks access keys and their last used date.
- Helps find unused or outdated identities.
- Review permissions and look for excessive access.

## 5. CHECK SERVICE ACCOUNTS

```
kubectl get serviceaccounts -A
kubectl describe serviceaccount <name>
cat /etc/systemd/system/*.service
```



- Lists Kubernetes service accounts.
- Review what each service account can access.
- Check systemd services and their user accounts.
- Look for hardcoded credentials or over-privileged accounts.

## 6. IDENTIFY STALE OR UNUSED ACCESS

```
sudo lastlog
aws iam get-credential-report
grep -i "password" /etc/shadow
```



- Find accounts that have not been used recently.
- Check for inactive IAM users and access keys.
- Look for accounts with no recent logins.
- Remove or disable unused accounts.

## 7. BEST PRACTICES

- ✓ Use least privilege access.
- ✓ Remove unused accounts and keys.
- ✓ Use SSH keys instead of passwords.
- ✓ Enforce MFA for cloud access.
- ✓ Regularly audit access and permissions.
- ✓ Keep a record of who has access and why.

## 8. COMMON RISKS



- Unused accounts can still be abused.
- Stolen keys can give full access.
- Over-privileged IAM roles increase risk.
- Service accounts may have excessive permissions.
- No MFA makes accounts easy to compromise.
- Lack of monitoring hides unauthorized access.



### JUNIOR ENGINEER TIP

Access auditing should be done regularly, not only during a handover. Always document who has access, why they have it, and when it was last used.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

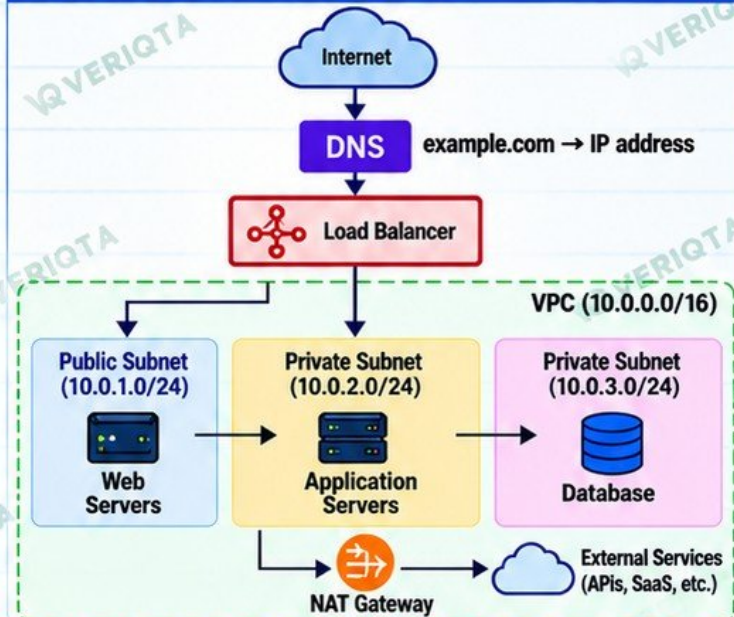
# Understanding the Existing Network Before Touching It

Discover private networks, subnets, routing, DNS, firewalls, gateways, and load balancers.

## 1. KEY NETWORK COMPONENTS

|                                                                                    |                                  |                                                  |
|------------------------------------------------------------------------------------|----------------------------------|--------------------------------------------------|
|   | <b>VPC / Network</b>             | Isolated network where your resources run.       |
|   | <b>Subnet</b>                    | Smaller network inside a VPC.                    |
|   | <b>Route Table</b>               | Decides where traffic goes.                      |
|   | <b>Internet Gateway</b>          | Connects your VPC to the internet.               |
|   | <b>NAT Gateway</b>               | Allows private resources to access the internet. |
|   | <b>Firewall / Security Group</b> | Controls what traffic is allowed.                |
|   | <b>Load Balancer</b>             | Distributes traffic to multiple servers.         |
|  | <b>DNS</b>                       | Converts domain names to IP addresses.           |

## 2. TYPICAL PRODUCTION NETWORK FLOW



## 3. DISCOVER NETWORK CONFIGURATION (LINUX)

|                                      |                                           |
|--------------------------------------|-------------------------------------------|
| <code>ip a</code>                    | Show network interfaces and IP addresses. |
| <code>ip route</code>                | Show routing table.                       |
| <code>cat /etc/resolv.conf</code>    | Show DNS configuration.                   |
| <code>ss -tulnp</code>               | Show open network ports.                  |
| <code>ping &lt;host&gt;</code>       | Test connectivity.                        |
| <code>traceroute &lt;host&gt;</code> | See the network path to a host.           |
| <code>dig &lt;domain&gt;</code>      | Query DNS records.                        |
| <code>nslookup &lt;domain&gt;</code> | Check DNS resolution.                     |

## 4. CLOUD NETWORK DISCOVERY (AWS EXAMPLE)

|                                                 |                         |
|-------------------------------------------------|-------------------------|
| <code>aws ec2 describe-vpcs</code>              | List all VPCs.          |
| <code>aws ec2 describe-subnets</code>           | List subnets.           |
| <code>aws ec2 describe-route-tables</code>      | Show route tables.      |
| <code>aws ec2 describe-internet-gateways</code> | List internet gateways. |
| <code>aws ec2 describe-nat-gateways</code>      | List NAT gateways.      |
| <code>aws ec2 describe-security-groups</code>   | Show security groups.   |
| <code>aws elbv2 describe-load-balancers</code>  | List load balancers.    |
| <code>aws route53 list-hosted-zones</code>      | List DNS hosted zones.  |

## 5. WHAT TO CHECK

- 1 What VPCs and networks exist?
- 2 What subnets are used and what resources are in them?
- 3 How is traffic routed (internet, VPN, peering)?
- 4 What firewalls or security groups control access?
- 5 What DNS records point to the environment?
- 6 What load balancers are used and what do they route to?

## 6. QUESTIONS TO ASK

- ✓ Is the environment public or private?
- ✓ Which systems must be accessible from the internet?
- ✓ Are there VPNs or on-premises connections?
- ✓ Who manages DNS and firewall rules?
- ✓ Are there multiple environments (dev, staging, prod)?
- ✓ Are there external services the application depends on?

## 7. COMMON MISTAKES

- ✗ Changing network rules without understanding impact.
- ✗ Assuming a server is public when it is private.
- ✗ Not checking DNS records.
- ✗ Missing hidden dependencies on external services.
- ✗ Overlooking security groups or firewall rules.



### JUNIOR ENGINEER TIP

Before making any changes, map the current network and understand how traffic flows. A small network change can break multiple services.



### PRODUCTION AWARENESS

Network misconfigurations can cause outages, security incidents, or expose internal systems. Always document what you find.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

07

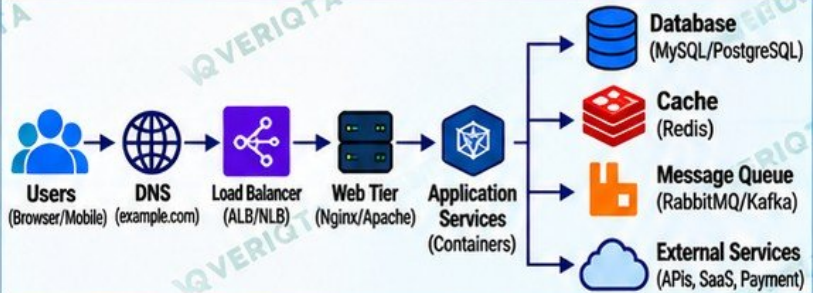
# Mapping How Production Applications Communicate

Trace application requests from entry points through services, databases, and external dependencies.

## 1. UNDERSTAND THE BIG PICTURE

- Identify all entry points (users, APIs, load balancers).
- Map how requests flow between services.
- Find all databases, caches, queues, and external services.
- Understand protocols and ports used.
- Document dependencies and data flow.

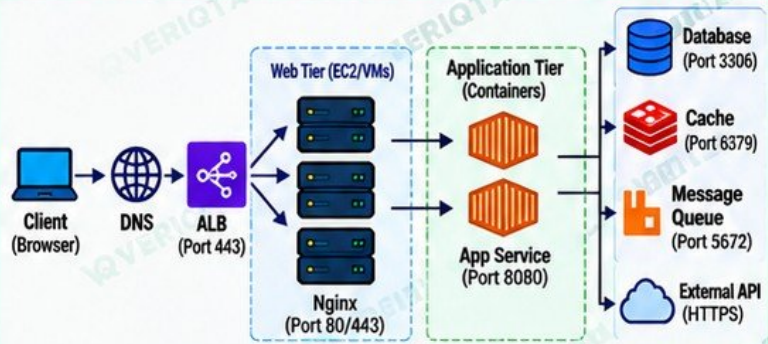
## 2. TYPICAL PRODUCTION APPLICATION FLOW



## 3. TRACE A REAL REQUEST (EXAMPLE)

- User enters a URL (example.com).
- DNS resolves the domain to an IP address.
- Request hits the load balancer.
- Load balancer forwards to web servers.
- Web server proxies to application service.
- Application service queries database.
- Application calls external services (e.g., payment API).
- Response flows back through the same path.

## 4. EXAMPLE: WEB APPLICATION REQUEST FLOW



## 5. TOOLS TO MAP COMMUNICATIONS

|                                                                                     |                                                                   |                                               |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------|-----------------------------------------------|
|  | <b>Networking tools</b><br>ping, traceroute, mtr, nc, curl        | Check connectivity and trace network path.    |
|  | <b>Log analysis</b><br>Access logs, application logs              | See real traffic and requests.                |
|  | <b>Service discovery</b><br>Kubernetes, service mesh, cloud tools | Find running services and endpoints.          |
|  | <b>Configuration review</b><br>Nginx, Apache, application configs | Understand routing rules and backends.        |
|  | <b>Cloud tools</b><br>AWS VPC, ALB, security groups               | Identify cloud network components.            |
|  | <b>Monitoring tools</b><br>Grafana, Datadog, CloudWatch           | View live traffic, metrics, and dependencies. |

## 6. USEFUL COMMANDS

|                                                |                                  |
|------------------------------------------------|----------------------------------|
| <code>nslookup example.com</code>              | Resolve domain to IP.            |
| <code>dig example.com</code>                   | Detailed DNS lookup.             |
| <code>ping &lt;host&gt;</code>                 | Test basic connectivity.         |
| <code>traceroute &lt;host&gt;</code>           | Trace network path (Linux).      |
| <code>mtr &lt;host&gt;</code>                  | Real-time network path.          |
| <code>curl -v https://example.com</code>       | See full request and response.   |
| <code>ss -tulnp</code>                         | Show listening ports.            |
| <code>netstat -tulnp</code>                    | Alternative to check open ports. |
| <code>kubectl get svc -A</code>                | List Kubernetes services.        |
| <code>kubectl get endpoints -A</code>          | Show service endpoints.          |
| <code>aws elbv2 describe-load-balancers</code> | List AWS load balancers.         |
| <code>aws ec2 describe-security-groups</code>  | Check security group rules.      |

## 7. DOCUMENT WHAT YOU FIND

- Draw a simple architecture diagram.
- List all services and their endpoints.
- Record ports, protocols, and health checks.
- Identify external dependencies and third-party services.
- Note any unknown or unexpected communication.

## 8. COMMON MISTAKES

- Missing hidden dependencies.
- Not checking DNS and external services.
- Assuming only internal communication.
- Ignoring non-standard ports.
- Overlooking third-party integrations.

## 9. JUNIOR ENGINEER TIP



Follow a single user request from start to finish. Write down every hop, service, and dependency. This helps you understand how everything fits together and where issues can occur.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Finding the Systems Nobody Told You About

Identify forgotten virtual machines, manual scripts, cron jobs, legacy applications, and shadow infrastructure.

## 1. WHAT ARE UNDOCUMENTED SYSTEMS?

- Systems that exist but are not in official documentation.
- Often created manually by previous engineers.
- May include old servers, scripts, databases, test environments, or third-party services.
- Can still be running and critical to production.

## 3. FIND VIRTUAL MACHINES AND INSTANCES

### AWS (Example)

```
aws ec2 describe-instances \
  --query "Reservations[*].Instances[*].[InstanceId,State,Tags]"
  --output table
```

### Azure (Example)

```
az vm list --output table
```

### GCP (Example)

```
gcloud compute instances list
```

- Lists all running and stopped instances.
- Look for old, unusual, or untagged instances.
- Check different regions and projects.

## 2. COMMON PLACES TO LOOK

  
**Cloud Accounts**  
(AWS, Azure, GCP)

  
**Virtual Machines**  
(Old/Unused)

  
**Manual Scripts**  
(Bash, Python)

  
**Cron Jobs**  
(Scheduled Tasks)

  
**Legacy Applications**  
(Old Versions)

  
**Databases**  
(Unknown/Unused)

  
**Shadow Infrastructure**  
(Outside IaC)

  
**External Services**  
(Third-Party)

## 4. FIND MANUAL SCRIPTS AND CRON JOBS

### Check user cron jobs

```
crontab -l
crontab -u <username> -l
```

### Check system-wide cron jobs

```
ls -la /etc/cron.d/
ls -la /etc/cron.daily/
ls -la /etc/cron.hourly/
ls -la /etc/cron.weekly/
ls -la /etc/cron.monthly/
```

### Find custom scripts

```
find / -type f -name "*.sh" 2>/dev/null
find / -type f -name "*.py" 2>/dev/null
```



- Look for scripts outside standard directories.
- Check for automation that is not in version control.
- Identify scripts owned by users who are no longer active.

## 5. FIND LEGACY APPLICATIONS

### Check running services

```
systemctl list-units --type=service --state=running
systemctl list-unit-files --type=service
```

- Look for old or unknown services.
- Check installed packages and versions.
- Look for applications running on non-standard ports.
- Review web servers for old applications.



## 6. FIND DATABASES AND OTHER SERVICES

### Check running databases

```
ss -tulnp | grep -E '3306|5432|27017|6379'
docker ps
kubectl get pods -A
systemctl list-units | grep -E 'mysql|postgres|mongo|redis'
```

- Find databases that are not documented.
- Check for local and remote database instances.
- Look for message queues, caches, and other supporting services.



## 7. CHECK FOR SHADOW INFRASTRUCTURE

- Review cloud accounts you may not normally use.
- Look for resources created outside Infrastructure as Code.
- Check for old test or staging environments still running.
- Identify external services (SaaS, APIs) used by applications.
- Talk to team members and review old tickets or emails.
- Look for untagged or poorly tagged resources.



## 8. USEFUL COMMANDS

|                                   |                                |
|-----------------------------------|--------------------------------|
| <code>ps aux</code>               | List running processes.        |
| <code>netstat -tulnp</code>       | Show open ports.               |
| <code>lsof -i</code>              | Show active connections.       |
| <code>docker ps -a</code>         | List all containers.           |
| <code>kubectl get all -A</code>   | List all Kubernetes resources. |
| <code>find / -name "*.sh"</code>  | Find shell scripts.            |
| <code>grep -R "password" /</code> | Find hardcoded credentials.    |

## 9. COMMON RISKS

- ✗ Old, unsupported systems.
- ✗ Scripts with hardcoded credentials.
- ✗ Test environments exposed to the internet.
- ✗ Missing monitoring and backups.
- ✗ Unknown dependencies.
- ✗ Security and compliance risks.



### JUNIOR ENGINEER TIP

Always verify why a system exists before stopping it. A system that looks unused might still be critical for a hidden process or external integration.



### REMEMBER

Undocumented systems are common in real production environments. Your job is to find them, understand them, document them, and raise any risks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

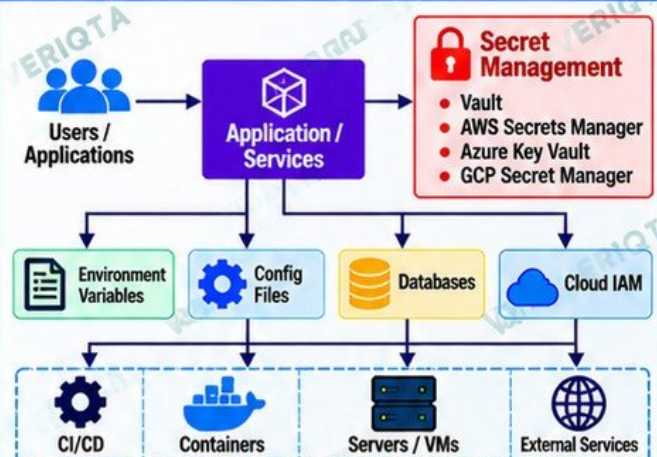
# Where Are the Secrets and Credentials Stored?

Locate credential management systems, understand access boundaries, and identify exposure risks without revealing secret values.

## 1. COMMON PLACES SECRETS ARE STORED

|                                                                                                                                                  |                                                                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  <b>Environment Variables</b><br>(Servers, Containers, CI/CD)   |  <b>Secret Management Tools</b><br>(HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, GCP Secret Manager) |
|  <b>Configuration Files</b><br>(App configs, .env files)        |  <b>Cloud Services</b><br>(IAM roles, access keys, service accounts)                                           |
|  <b>Databases</b><br>(Connection strings, API keys)             |  <b>Containers &amp; Orchestration</b><br>(Kubernetes secrets, Docker)                                         |
|  <b>Source Code Repositories</b><br>(GitHub, GitLab, Bitbucket) |  <b>Manual Storage</b><br>(Scripts, spreadsheets, emails, chat tools)                                          |
|  <b>CI/CD Systems</b><br>(Jenkins, GitHub Actions, GitLab CI)   |  <b>Operating System</b><br>(Keyrings, credential stores)                                                      |

## 2. EXAMPLE ARCHITECTURE



## 3. FIND ENVIRONMENT VARIABLES

```
printenv | sort
env | grep -iE 'key|secret|token|pass'
cat /proc/1/environ 2>/dev/null | tr '\0' '\n'
```



- Lists all environment variables.
- Look for variables that may contain credentials.
- Check application processes and containers.

## 4. CHECK CONFIGURATION FILES

```
grep -rEi 'password|secret|token|key' /etc/
grep -rEi 'password|secret|token|key' /opt/
find / -type f -name "*.conf" -o -name "*.env" 2>/dev/null
```



- Search for potential secrets in config files.
- Check .env files and application configs.
- Look in common directories and custom application paths.
- Do not open or expose actual secret values.

## 5. CHECK CONTAINERS AND KUBERNETES

```
docker inspect <container> | grep -i env
kubectl get secrets -A
kubectl describe secret <name> -n <namespace>
```



- Inspect container environment variables.
- List Kubernetes secrets (metadata only).
- Check mounted secret files.
- Understand which applications use the secrets.

## 6. CHECK SECRET MANAGEMENT TOOLS

|                                                                                                         |                                                                                                           |
|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
|  HashiCorp Vault     |  GCP Secret Manager  |
|  AWS Secrets Manager |  Kubernetes Secrets  |
|  Azure Key Vault     |  CI/CD Secret Stores |

- Identify which secret management tools are used.
- Check who has access to them.
- Review policies, roles, and access boundaries.
- Do not retrieve or display actual secret values.

## 7. CHECK SOURCE CODE REPOSITORIES

```
git grep -iE 'password|secret|token|api_key'
git log -p | grep -iE 'password|secret|token'
trufflehog . # (if installed)
```



- Search for hardcoded credentials.
- Check commit history for accidental exposure.
- Scan all branches and private repositories.
- Use secret scanning tools where available.

## 8. COMMON RISKS



- Hardcoded credentials in code.
- Overly permissive access to secrets.
- Secrets stored in plain text.
- Stale or unused credentials.
- Credentials in old servers or scripts.
- Secrets exposed in logs or monitoring.
- No access boundaries or audit trails.
- Shared credentials between environments.

## 9. BEST PRACTICES

- ✓ Use a dedicated secret manager.
- ✓ Apply least privilege access.
- ✓ Rotate credentials regularly.
- ✓ Remove unused or stale credentials.
- ✓ Enforce access boundaries.
- ✓ Audit who can access secrets.
- ✓ Use environment-specific secrets.
- ✓ Enable logging and monitoring for access.



### JUNIOR ENGINEER TIP

Your job is to find where secrets are stored and who can access them, not to read or expose the actual values. Understanding secret storage and access boundaries helps you secure the environment and prevent accidental leaks.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

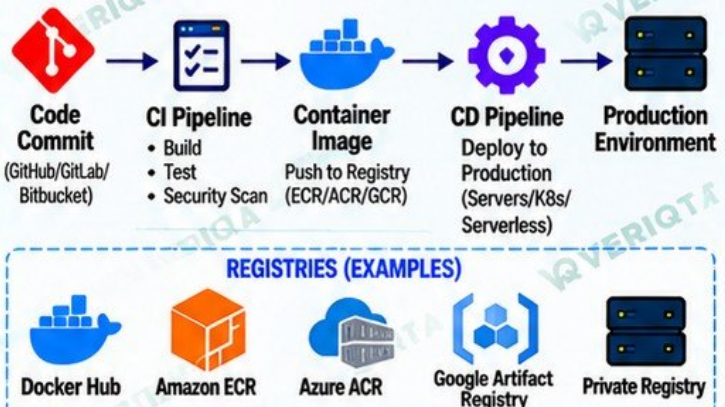
# Understanding How Code Reaches Production

Discover repositories, CI/CD pipelines, deployment methods, registries, approvals, and rollback procedures.

## 1. KEY COMPONENTS IN THE PATH TO PRODUCTION

|                                                                                    |                               |                                                                  |
|------------------------------------------------------------------------------------|-------------------------------|------------------------------------------------------------------|
|   | <b>Source Code Repository</b> | Stores application code, configuration, and infrastructure code. |
|   | <b>CI/CD Pipeline</b>         | Builds, tests and deploys code automatically or manually.        |
|   | <b>Container Registry</b>     | Stores container images (Docker, ECR, ACR, GCR, etc.).           |
|   | <b>Deployment Method</b>      | Deploys to servers, Kubernetes, or serverless platforms.         |
|   | <b>Approvals</b>              | Manual or automated checks before production release.            |
|  | <b>Rollback Procedure</b>     | Returns to a previous stable version when issues occur.          |

## 2. TYPICAL CI/CD FLOW TO PRODUCTION



## 3. FIND REPOSITORIES AND BRANCH STRATEGY

```
# List repositories (GitHub CLI)
gh repo list
# Clone a repository
git clone https://github.com/org/repo.git
# Check branches
git branch -a
# View recent commits
git log --oneline --graph -n 10
```



- Identify all repositories used in production.
- Understand branch strategy (main, release, develop).
- Check who has access and recent activity.
- Look for infrastructure code repositories (Terraform, Helm, etc).

## 4. EXAMPLE DEPLOYMENT METHODS

|                                                                                     |                            |                                                     |
|-------------------------------------------------------------------------------------|----------------------------|-----------------------------------------------------|
|  | <b>Kubernetes (K8s)</b>    | Deploy using manifests, Helm or ArgoCD.             |
|  | <b>Containers</b>          | Pull image from registry and run on servers or K8s. |
|  | <b>Serverless</b>          | Deploy using AWS Lambda, API Gateway, etc.          |
|  | <b>Traditional Servers</b> | Deploy using scripts, Ansible, or manual process.   |
|  | <b>Cloud Services</b>      | Deploy to AWS, Azure or GCP managed services.       |

## 5. CHECK CI/CD PIPELINES

```
# GitHub Actions
cat .github/workflows/deploy.yml
# GitLab CI
cat .gitlab-ci.yml
# Jenkins (view pipeline)
# Go to Jenkins UI -> Job -> Pipeline
# List recent builds (Jenkins CLI)
jenkins-cli list-builds <job-name>
```



- Identify all pipelines and what they do.
- Understand build, test, and deploy stages.
- Check triggers (push, merge, manual).
- Look for environment-specific configurations.

## 6. IDENTIFY APPROVALS AND CONTROLS

|                                                                                     |                          |                                                 |
|-------------------------------------------------------------------------------------|--------------------------|-------------------------------------------------|
|  | <b>Manual Approvals</b>  | Who approves production releases?               |
|  | <b>Change Management</b> | Is there a formal change process?               |
|  | <b>Security Scans</b>    | Are there code, dependency or container scans?  |
|  | <b>Access Controls</b>   | Who can deploy to production?                   |
|  | <b>Audit Logs</b>        | Are deployment activities logged and monitored? |

## 7. UNDERSTAND ROLLBACK PROCEDURES

|                                                                                       |                                |                                                           |
|---------------------------------------------------------------------------------------|--------------------------------|-----------------------------------------------------------|
|  | <b>Previous Version</b>        | How to return to the last stable release?                 |
|  | <b>Image Rollback</b>          | Use previous container image tag.                         |
|  | <b>K8s Rollback</b>            | <code>kubectl rollout undo deployment/&lt;name&gt;</code> |
|  | <b>Infrastructure Rollback</b> | Use IaC (Terraform) or configuration management.          |
|  | <b>Database Rollback</b>       | Use snapshots or point-in-time recovery.                  |

## 8. QUESTIONS TO ASK

- Where is the source code stored?
- How are builds and deployments triggered?
- What environments exist (dev, staging, prod)?
- Who can approve and deploy to production?
- How do we roll back if something fails?

## 9. COMMON RISKS

- Untracked or undocumented repositories.
- Manual deployments with no audit logs.
- No automated tests or security scans.
- No rollback plan or tested rollback process.
- Too many people with deployment access.
- Hardcoded credentials in pipelines.

## 10. JUNIOR ENGINEER TIP



Take time to understand the entire flow from code commit to production. Do not change pipelines or deployment processes until you fully understand how they work.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Identifying Single Points of Failure

Examine infrastructure components whose failure could interrupt critical services.

## 1. WHAT IS A SINGLE POINT OF FAILURE?



A component whose failure can cause a complete or major service outage.



Can be hardware, software, network, people, process or external service.

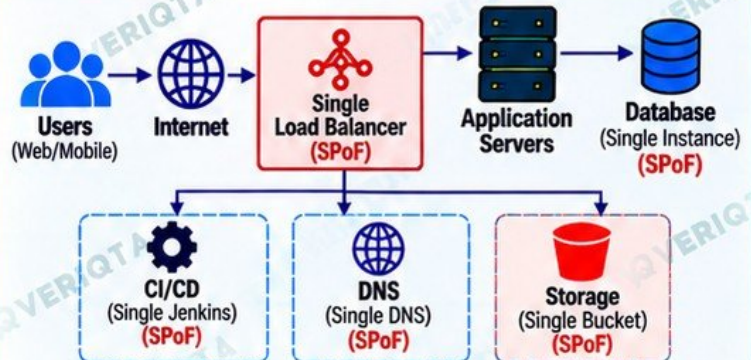


Reduces system resilience and availability.



Must be identified and mitigated.

## 2. EXAMPLE ARCHITECTURE WITH SPOF



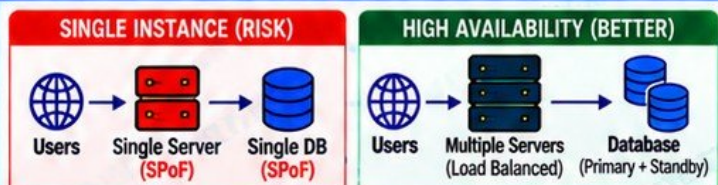
## 3. COMMON SINGLE POINTS OF FAILURE

|                                                                                     |                    |                                                          |
|-------------------------------------------------------------------------------------|--------------------|----------------------------------------------------------|
|  | Load Balancer      | Only one load balancer handling all traffic.             |
|  | Database           | Single database instance with no replication.            |
|  | Application Server | Only one server or container running the application.    |
|  | DNS                | Single DNS provider or zone.                             |
|  | Network Component  | Single firewall, router or NAT gateway.                  |
|  | Storage            | Single storage bucket or file server.                    |
|  | CI/CD Pipeline     | Only one CI/CD server or pipeline.                       |
|  | External Service   | Relying on a single third-party API or payment provider. |
|  | Key Person         | Critical knowledge owned by one person.                  |

## 4. HOW TO IDENTIFY SPOFS

- 1 Map your architecture and dependencies.
- 2 Identify components with no redundancy.
- 3 Check for single instances (servers, databases, services).
- 4 Review network design (single gateways, firewalls, DNS).
- 5 Check external service dependencies.
- 6 Look for manual or human dependencies.
- 7 Validate monitoring and alerting coverage.
- 8 Test failure scenarios or review past incidents.

## 5. EXAMPLE: HIGH AVAILABILITY VS SINGLE INSTANCE



## 6. TOOLS AND COMMANDS TO CHECK

|                   |                                   |
|-------------------|-----------------------------------|
| nmap              | Discover live hosts and services. |
| dig / nslookup    | Check DNS configuration.          |
| curl / nc         | Test external dependencies.       |
| kubectl get nodes | Check Kubernetes node count.      |
| aws / az / gcloud | Review cloud resource redundancy. |
| ss -tulnp         | Check open ports and services.    |
| systemctl status  | Verify service status.            |
| traceroute / mtr  | Trace network paths.              |

## 7. QUESTIONS TO ASK

- 1 Is there more than one instance?
- 2 What happens if this component fails?
- 3 Do we have redundancy or failover?
- 4 Are there alternative providers?
- 5 Is the data replicated and backed up?
- 6 Are there multiple availability zones?
- 7 What is the recovery time objective (RTO)?
- 8 What is the recovery point objective (RPO)?

## 8. MITIGATION STRATEGIES

- 1 Use multiple instances and load balancers.
- 2 Implement database replication.
- 3 Deploy across multiple availability zones or regions.
- 4 Use managed services with automatic failover.
- 5 Set up regular backups and disaster recovery.
- 6 Remove single person dependencies.
- 7 Implement monitoring and alerting.
- 8 Test failover and recovery procedures.



### JUNIOR ENGINEER TIP

Always look for components that have no backup or alternative. If you cannot clearly explain what happens when a component fails, treat it as a potential single point of failure.



### REMEMBER

A single point of failure can turn a minor issue into a major outage. Identify, document and help remove or reduce them.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Who Owns What in Production?

Establish service ownership, escalation contacts, infrastructure responsibilities, and operational handover gaps.

## 1. WHAT IS SERVICE OWNERSHIP?



A clearly defined person or team responsible for a service in production.



Accountable for uptime, performance, security, changes and incidents.

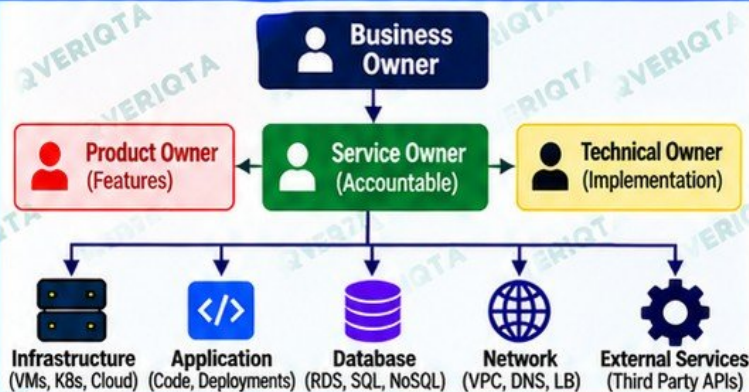


Includes application, infrastructure, dependencies and data.



Ensures there are escalation contacts and proper handover documentation.

## 2. EXAMPLE SERVICE OWNERSHIP MAP



## 3. KEY AREAS TO IDENTIFY OWNERSHIP

|                                                                                     |                   |                                                     |
|-------------------------------------------------------------------------------------|-------------------|-----------------------------------------------------|
|  | Infrastructure    | Servers, VMs, Kubernetes clusters, cloud resources. |
|  | Applications      | Web apps, APIs, background jobs, batch processes.   |
|  | Databases         | Primary, read replicas, backups, data pipelines.    |
|  | Networking        | VPC, subnets, DNS, firewalls, load balancers.       |
|  | External Services | Third-party APIs, SaaS tools, payment providers.    |
|  | Security          | IAM, certificates, secrets, access controls.        |
|  | Operational Tasks | Monitoring, backups, patching, incident response.   |

## 4. QUESTIONS TO FIND OWNERSHIP

- Who is the owner of this service?
- Who maintains the infrastructure?
- Who can approve changes?
- Who is on call for incidents?
- Who manages the database?
- Who owns the DNS and networking?
- Who manages third-party integrations?
- What happens if the current owner is unavailable?
- Is there documentation for handover?



## 5. EXAMPLE ESCALATION PATH



## 6. DOCUMENT OWNERSHIP INFORMATION

- ✓ Create a service ownership register.
- ✓ List all services and their owners.
- ✓ Include escalation contacts (email, phone, chat).
- ✓ Document infrastructure responsibilities.
- ✓ Identify single points of ownership.
- ✓ Keep information updated and accessible.
- ✓ Record operational handover notes.



## 7. COMMON GAPS TO LOOK FOR

- ✗ No clear service owner.
- ✗ Outdated contact information.
- ✗ Knowledge only in one person.
- ✗ No handover documentation.
- ✗ Unclear infrastructure ownership.
- ✗ Third-party services without owners.
- ✗ Missing escalation path.
- ✗ Teams that are no longer in the company.

## 8. TOOLS AND PLACES TO CHECK

-  Internal Documentation (Confluence, Wiki)
-  Incident Management (PagerDuty, Opsgenie)
-  Cloud Provider IAM (AWS, Azure, GCP)
-  Infrastructure as Code (Terraform, CloudFormation)
-  Communication Tools (Slack, Teams)



### JUNIOR ENGINEER TIP

Always find out who owns each service before making changes. If you are not sure, ask. This helps you avoid incidents and ensures the right people are involved.



### REMEMBER

Unclear ownership leads to delayed response, miscommunication, and outages. Identify gaps and escalate them so ownership is clear and up to date.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Understanding the Existing Monitoring and Alerting Setup

Find dashboards, metrics, logs, alerts, notification channels, and monitoring blind spots.

## 1. WHAT TO LOOK FOR



**Existing monitoring tools**  
(e.g., Prometheus, Grafana, CloudWatch, Datadog).



**Dashboards and key metrics**  
(infrastructure, application, business).



**Log collection and analysis**  
(e.g., ELK, Loki, CloudWatch Logs).



**Alert rules and thresholds**  
(what triggers an alert).

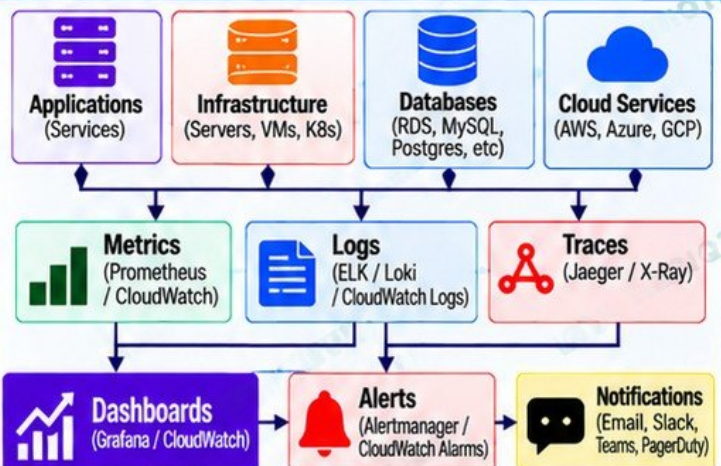


**Notification channels**  
(email, Slack, Teams, PagerDuty, etc.).



**Missing coverage or blind spots**  
(services, environments, or critical metrics not monitored).

## 2. EXAMPLE MONITORING ARCHITECTURE



## 3. CHECK EXISTING DASHBOARDS

```
# list Grafana dashboards (API example)"
curl -H "Authorization: Bearer <TOKEN>" \
https://grafana.example.com/api/search
```



```
# check AWS CloudWatch dashboards
aws cloudwatch list-dashboards
```



```
# view Kubernetes monitoring components
kubectl get pods -n monitoring
```

- Identify existing dashboards for infrastructure, applications and business metrics.
- Understand who uses them and what they monitor.
- Check for missing dashboards or outdated information.

## 4. CHECK METRICS AND ALERTS

```
# Prometheus: view active alert rules
curl http://prometheus:9090/api/v1/rules
```



```
# Alertmanager: check active alerts
curl http://alertmanager:9093/api/v1/alerts
```



```
# CloudWatch: list alarms
aws cloudwatch describe-alarms
```



- Review existing alert rules and thresholds.
- Check which metrics are being collected.
- Identify critical alerts and who gets notified.
- Look for missing or incorrect alerting (e.g., no alerts for databases).

## 5. CHECK LOGGING SETUP

```
# Check Elasticsearch indices
curl -X GET "http://elasticsearch:9200/_cat/indices?v"
```



```
# Check Loki log sources
curl "http://loki:3100/loki/api/v1/labels"
```



```
# Check CloudWatch log groups
aws logs describe-log-groups
```



- Find where application and infrastructure logs are stored.
- Check log retention period.
- Verify logs are being collected from all critical systems.
- Look for missing logs or gaps in coverage.

## 6. CHECK NOTIFICATION CHANNELS

```
# Check Alertmanager receivers
curl http://alertmanager:9093/api/v1/receivers
```



```
# Check PagerDuty services (API example)"
curl -H "Authorization: Token <TOKEN>" \
https://api.pagerduty.com/services
```



- Identify which channels are used for notifications.
- Verify the right people and teams are receiving alerts.
- Check for unused or outdated notification channels.
- Confirm escalation paths (on-call schedule, PagerDuty, etc).

## 7. IDENTIFY MONITORING BLIND SPOTS

- ✓ Critical services without metrics or alerts.
- ✓ Databases or infrastructure not monitored.
- ✓ Applications with missing logs or traces.
- ✓ Environments (dev, staging, prod) with different coverage.
- ✓ Third-party services or external dependencies not monitored.
- ✓ Alerts that are noisy, missing, or not actionable.



## 8. QUESTIONS TO ASK

- ? What tools are used for monitoring and alerting?
- ? What metrics and logs are collected?
- ? Who maintains the dashboards and alert rules?
- ? Who receives alerts and how are they escalated?
- ? Are there any services without monitoring?
- ? How are alerts tested and updated?
- ? Are there any known blind spots?
- ? When was the last review of the monitoring setup?



## 9. JUNIOR ENGINEER TIPS

- ✓ Explore existing dashboards.
- ✓ Understand key metrics for each service.
- ✓ Check where logs are stored and retained.
- ✓ Identify critical alerts and who gets them.
- ✓ Document any missing monitoring.
- ✓ Ask questions and verify your findings with the team.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Investigating Backup and Disaster Recovery Readiness

Identify backup locations, retention policies, recovery objectives, and evidence of restore testing.

## 1. WHY BACKUP AND DR MATTERS

-  **Protects against data loss** (accidental deletion, corruption, ransomware).
-  **Ensures business continuity** during system or cloud failures.
-  **Helps meet compliance and regulatory requirements.**
-  **Confirms you can recover within acceptable time and data loss limits.**

## 3. KEY THINGS TO INVESTIGATE

|                                                                                     |                               |                                                                 |
|-------------------------------------------------------------------------------------|-------------------------------|-----------------------------------------------------------------|
|   | <b>Backup Locations</b>       | Where are backups stored?<br>Onsite, offsite, cross-region?     |
|  | <b>Retention Policies</b>     | How long are backups kept?<br>Daily, weekly, monthly, yearly?   |
|  | <b>Recovery Objectives</b>    | What are the RTO (time) and RPO (data loss) targets?            |
|  | <b>Restore Testing</b>        | Is there evidence of successful restore tests? How often?       |
|  | <b>Coverage Scope</b>         | What systems, databases and applications are included/excluded? |
|  | <b>Access Control</b>         | Who can create, delete or modify backups?                       |
|  | <b>Disaster Recovery Plan</b> | Is there a documented DR plan?<br>Is it up to date and tested?  |

## 5. USEFUL COMMANDS (EXAMPLES)

|                                            |                               |
|--------------------------------------------|-------------------------------|
| <code>aws backup list-backup-jobs</code>   | List backup jobs (AWS).       |
| <code>aws rds describe-db-snapshots</code> | List RDS snapshots (AWS).     |
| <code>az backup item list</code>           | List backup items (Azure).    |
| <code>gcloud compute snapshots list</code> | List snapshots (GCP).         |
| <code>kubectl get volumesnapshot -A</code> | List K8s volume snapshots.    |
| <code>ls -lh /backup</code>                | Check backup files on server. |
| <code>cat /etc/backup/backup.conf</code>   | View backup configuration.    |

-  Use the appropriate CLI for your environment (AWS, Azure, GCP, etc.).
-  Check backup schedules and last successful run time.
-  Verify storage usage and retention settings.
-  Review logs for failed backup jobs.

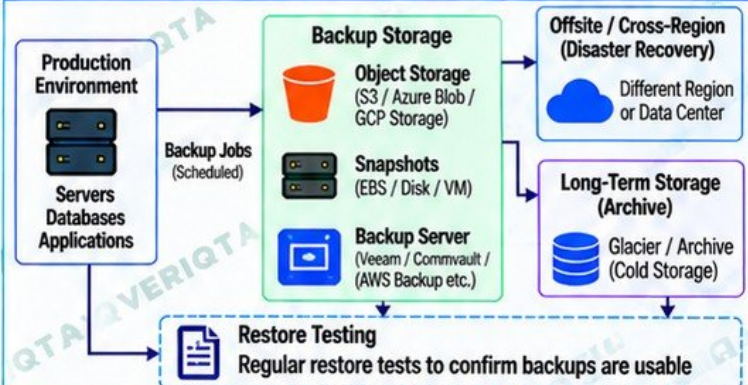
## 8. QUESTIONS TO ASK

- What systems and data are backed up?
- Where are the backups stored?
- How long are backups retained?
- When was the last successful restore test?
- What are the RTO and RPO targets?
- Who is responsible for backups and DR?
- Are backups monitored and alerted?
- What is the disaster recovery plan and when was it last tested?
- Are there any data or systems not included in backups?

## 9. SIGNS OF GOOD READINESS

- Regular successful backup jobs.
- Backups stored offsite and encrypted.
- Clear retention policies.
- Documented and tested restore procedures.
- Defined RTO and RPO.
- Monitoring and alerts for backup failures.
- Up-to-date disaster recovery plan.
- Assigned ownership and escalation contacts.
- Evidence of recent restore testing.

## 2. EXAMPLE BACKUP AND DR ARCHITECTURE



## 4. COMMON BACKUP SOLUTIONS

|                                                                                     |                            |                                                                                      |
|-------------------------------------------------------------------------------------|----------------------------|--------------------------------------------------------------------------------------|
|  | <b>AWS Backup</b>          | Centralized backup for AWS services (EC2, RDS, EFS, etc.).                           |
|  | <b>Azure Backup</b>        | Backup and site recovery for Azure resources.                                        |
|  | <b>Google Cloud Backup</b> | Backup and DR for GCP resources.                                                     |
|  | <b>Veeam</b>               | Popular third-party backup solution (on-prem and cloud).                             |
|  | <b>Commvault</b>           | Enterprise backup and recovery platform.                                             |
|  | <b>Snapshots</b>           | Point-in-time copies of disks, volumes or databases.                                 |
|  | <b>Object Storage</b>      | Low-cost, durable storage for long-term backups (AWS S3, Glacier, Azure Blob, etc.). |

## 6. COMMON RISKS

-  No backups or incomplete coverage.
-  Backups stored in the same region as production.
-  Short or unclear retention policies.
-  No regular restore testing.
-  Backups accessible by unauthorized users.
-  RTO and RPO not defined.
-  Outdated disaster recovery plan.
-  Backup failures not monitored.
-  Ransomware encrypting backups.
-  Dependency on a single backup tool or provider.

## 7. BEST PRACTICES

-  Keep backups in a different region or location.
-  Follow the 3-2-1 backup rule.
-  Define and document RTO and RPO.
-  Test restores regularly and keep evidence.
-  Automatically monitor backup jobs and failures.
-  Use encryption and strict access control.
-  Store long-term backups in cold storage.
-  Include all critical systems and data.
-  Keep the disaster recovery plan updated.
-  Review backup strategy during infrastructure changes.

## 10. JUNIOR ENGINEER TIP

-  Always verify backups before any major change.
-  Make sure you know how to restore a system, not just how to back it up.
-  Check backup status regularly.
-  Keep notes of what is backed up and where.
-  If you are not sure, ask. Backup and DR are critical for production.
-  Document what you find and share with your team.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Reading the Environment's Incident History

Examine previous outages, recurring failures, incident reports, unresolved issues, and operational workarounds.

## 1. WHY INCIDENT HISTORY MATTERS



Helps you understand common failure patterns.



Reveals weak points in the environment.



Shows how incidents are handled and if they are properly resolved.

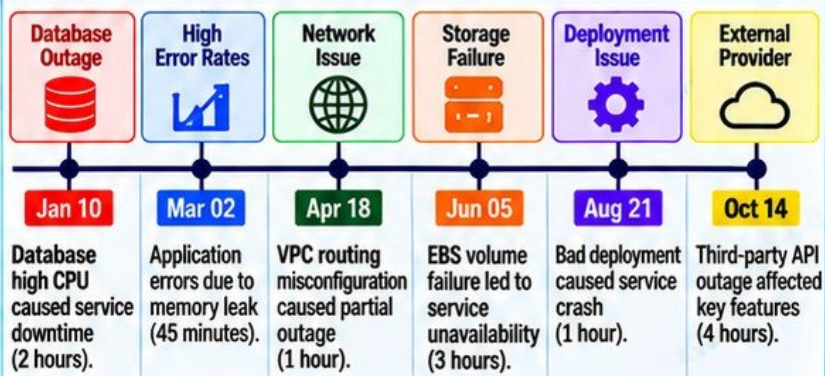


Helps you make better decisions and prevent future outages.



Identifies technical, process and communication gaps.

## 2. EXAMPLE INCIDENT HISTORY TIMELINE



## 3. WHAT TO LOOK FOR IN INCIDENT REPORTS



Root Cause

What actually caused the incident?



Impact

Which services were affected?  
How long did it last?



Detection

How was the issue detected?  
(Alerts, users, monitoring, etc.)



Response

What actions were taken?  
How long did it take to respond?



Resolution

How was it fixed? Was it a permanent fix?



Recurrence

Has this issue happened before?



Unresolved Issues

Are there open action items or known risks?



Workarounds

Are there temporary fixes still in use?

## 4. REAL EXAMPLES OF RECURRING INCIDENTS

| Incident Type            | Example                             | Possible Cause                     | What to Check                             |
|--------------------------|-------------------------------------|------------------------------------|-------------------------------------------|
| High CPU                 | Frequent CPU spikes on app server   | Inefficient code or traffic spikes | Metrics, logs, scaling policy             |
| Database Connectivity    | Intermittent DB connection failures | Network issues, DB overload        | DB logs, network routing, security groups |
| Disk Space Full          | Servers running out of disk space   | No monitoring or log rotation      | Disk usage, log configuration             |
| Service Crashes          | Application often restarts          | Memory leaks or bad deployments    | App logs, deployment history              |
| Third-Party API Failures | Payment API outages                 | External provider instability      | Status pages, retry logic, fallbacks      |
| Certificate Expiry       | Expired SSL certificates            | No automated renewal               | Certificate dates, monitoring alerts      |

## 5. USEFUL COMMANDS AND PLACES TO CHECK

|                                               |                                  |
|-----------------------------------------------|----------------------------------|
| <code>kubectl get events -A</code>            | Check Kubernetes events.         |
| <code>kubectl describe pod &lt;pod&gt;</code> | View pod failure details.        |
| <code>journalctl -xe</code>                   | Check system logs (Linux).       |
| <code>grep -i error /var/log/*</code>         | Search for error logs.           |
| <code>aws cloudwatch logs</code>              | View AWS CloudWatch logs.        |
| <code>aws health describe-events</code>       | Check AWS service health events. |
| <code>cat /var/log/nginx/error.log</code>     | Check web server errors.         |
| <code>tail -n 100 /var/log/syslog</code>      | View recent system logs.         |



## 6. QUESTIONS TO ASK

- 1 What were the biggest incidents in the last 12 months?
- 2 How often do these incidents occur?
- 3 What are the most common root causes?
- 4 Are there any unresolved issues?
- 5 What workarounds are currently in place?
- 6 Are there open action items from past incidents?
- 7 How are incidents documented and shared?
- 8 Is there a post-incident review process?
- 9 Who was involved in the last major incident?
- 10 What has been done to prevent recurrence?



## 7. COMMON RED FLAGS

- Repeated incidents with the same root cause.
- Unresolved or overdue action items.
- No post-incident reviews.
- Frequent use of temporary workarounds.
- Missing or incomplete incident documentation.
- Lack of monitoring for known failure points.

## 8. BEST PRACTICES

- Review at least 6-12 months of incident history.
- Identify trends and recurring patterns.
- Ensure action items are tracked and closed.
- Validate that monitoring and alerting exist.
- Reduce reliance on manual workarounds.
- Use incident history to improve reliability and prevent future outages.

## 9. JUNIOR ENGINEER TIP



Read past incident reports even if you did not work on them. It will help you understand the environment, recognize common issues, and avoid repeating the same mistakes. Always ask questions if anything is unclear.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

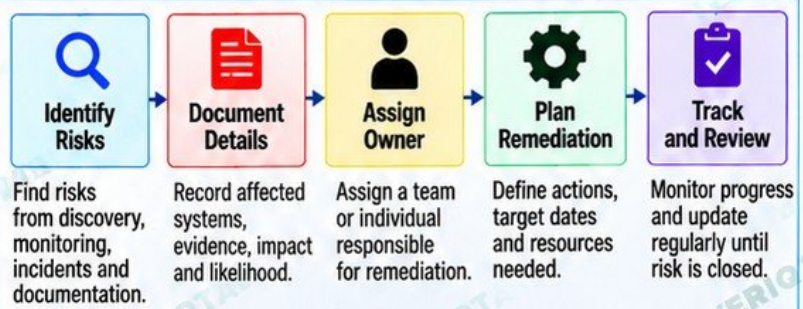
# Building Your First Production Risk Register

Document discovered risks, affected systems, supporting evidence, owners, and proposed remediation.

## 1. WHAT IS A RISK REGISTER?

-  A centralized record of risks in your production environment.
-  Helps you understand what can go wrong and take action early.
-  Includes details like affected systems, impact, likelihood, owners and remediation.
-  Used by engineering, operations, security and management.
-  Keeps everyone aligned and ensures risks are tracked to closure.

## 2. PRODUCTION RISK REGISTER WORKFLOW



## 3. EXAMPLE PRODUCTION RISK REGISTER

| ID    | Risk                                    | Affected Systems | Impact   | Likelihood | Evidence               | Owner         | Remediation                     | Status      |
|-------|-----------------------------------------|------------------|----------|------------|------------------------|---------------|---------------------------------|-------------|
| R-001 | Single point of failure (load balancer) | Web application  | High     | Medium     | Architecture diagram   | Platform Team | Deploy multi-AZ load balancer   | Open        |
| R-002 | No database backups                     | Production RDS   | Critical | High       | No backup policy found | Database Team | Enable backups and test restore | Open        |
| R-003 | Expired SSL certificate                 | API service      | High     | Medium     | Monitoring alert       | DevOps Team   | Renew certificate and automate  | In Progress |
| R-004 | Outdated server OS                      | Legacy VM        | Medium   | High       | Version check output   | Infra Team    | Upgrade to supported version    | Open        |
| R-005 | No incident runbook                     | Payment service  | High     | Medium     | No runbook found       | Service Owner | Create and review runbook       | Open        |

## 4. KEY INFORMATION TO INCLUDE

|                                                                                       |                             |                                                 |
|---------------------------------------------------------------------------------------|-----------------------------|-------------------------------------------------|
|  | <b>Risk Description</b>     | Clear and specific explanation of the risk.     |
|  | <b>Affected Systems</b>     | Applications, infrastructure, services or data. |
|  | <b>Impact</b>               | What happens if the risk materializes.          |
|  | <b>Likelihood</b>           | Probability of it happening (High/Medium/Low).  |
|  | <b>Supporting Evidence</b>  | Logs, screenshots, documents or observations.   |
|  | <b>Owner</b>                | Team or individual responsible for remediation. |
|  | <b>Proposed Remediation</b> | Action plan to reduce or eliminate the risk.    |
|  | <b>Status</b>               | Open, In Progress, Mitigated or Closed.         |

## 5. RISK SEVERITY MATRIX

| Likelihood | Impact |        |        |          |
|------------|--------|--------|--------|----------|
|            | Low    | Medium | High   | Critical |
| High       | Medium | High   | High   | Critical |
| Medium     | Low    | Medium | High   | High     |
| Low        | Low    | Low    | Medium | High     |

## 6. COMMON PRODUCTION RISKS

-  Single points of failure.
-  Missing or untested backups.
-  Outdated systems and software.
-  Misconfigured cloud resources.
-  Weak access controls.
-  Unmonitored critical services.
-  Missing documentation and runbooks.
-  Dependency on third-party services.

## 7. USEFUL COMMANDS FOR EVIDENCE

|                                               |                         |
|-----------------------------------------------|-------------------------|
| <code>aws rds describe-db-instances</code>    | Check RDS configuration |
| <code>aws backup list-backup-jobs</code>      | Check backup jobs       |
| <code>kubectl get nodes</code>                | Check cluster nodes     |
| <code>kubectl get pods -A</code>              | Check running workloads |
| <code>ss -tulnp</code>                        | Check open ports        |
| <code>curl -I https://service</code>          | Check SSL certificate   |
| <code>systemctl status &lt;service&gt;</code> | Check service status    |
| <code>df -h</code>                            | Check disk usage        |
| <code>free -h</code>                          | Check memory usage      |
| <code>cat /etc/os-release</code>              | Check OS version        |

## 8. QUESTIONS TO ASK

- What are the biggest risks we have today?
- Which systems would be most affected?
- What is the likelihood and potential impact?
- Do we have evidence for each risk?
- Who owns the risk and is the owner still correct?
- What is the current remediation plan?
- Are there any overdue or unaddressed risks?
- How often is the risk register reviewed?
- Are new risks being added from incidents or changes?
- What risks can be mitigated quickly?

## 9. BEST PRACTICES

- Keep the risk register simple and practical.
- Use clear and consistent risk categories.
- Include real evidence, not assumptions.
- Assign owners and target remediation dates.
- Review and update regularly (e.g., monthly).
- Link risks to incidents, changes and audits.
- Close risks only after verification.
- Use the register to drive action, not just reporting.

## JUNIOR ENGINEER TIP

-  Start small and document what you find.
- Use screenshots, logs or commands as evidence.
- Be honest about unknowns.
- Ask your team for help to understand impact and likelihood.
- Keep the register updated as you learn more about the environment.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# What Should You Fix First, and What Should You Leave Alone?

Understand change risk, maintenance windows, approval requirements, dependencies, and remediation planning.

## 1. FACTORS THAT DETERMINE WHAT TO FIX FIRST

|                                                                                   |                              |                                                               |
|-----------------------------------------------------------------------------------|------------------------------|---------------------------------------------------------------|
|  | <b>Business Impact</b>       | Fix issues that affect critical services and customers first. |
|  | <b>Risk Level</b>            | Prioritize high risk and high likelihood issues.              |
|  | <b>Security Exposure</b>     | Address vulnerabilities that could be actively exploited.     |
|  | <b>Operational Stability</b> | Fix issues causing frequent incidents or instability.         |
|  | <b>Ease of Remediation</b>   | Consider quick wins that reduce significant risk.             |
|  | <b>Dependencies</b>          | Understand what other systems are affected by the change.     |

## 2. WHEN TO FIX VS WHEN TO LEAVE ALONE

|  <b>FIX FIRST</b>                                                                                                                                                                                                                                              |  <b>LEAVE ALONE FOR NOW</b>                                                                                                                                                                                                                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>✓ Critical service outages</li> <li>✓ Actively exploited security risks</li> <li>✓ Data loss or corruption risks</li> <li>✓ High likelihood and high impact issues</li> <li>✓ Compliance violations</li> <li>✓ Simple fixes with low change risk</li> <li>✓ Issues causing frequent incidents</li> </ul> | <ul style="list-style-type: none"> <li>✓ Low risk cosmetic issues</li> <li>✓ Changes during freeze period</li> <li>✓ Systems with high change risk and no current impact</li> <li>✓ Components with critical dependencies</li> <li>✓ Issues requiring major refactoring</li> <li>✓ Non-urgent upgrades</li> <li>✓ Experimental changes</li> </ul> |

## 3. CHANGE RISK ASSESSMENT

| Risk Level    | Description                          | Examples                                           | Action                                               |
|---------------|--------------------------------------|----------------------------------------------------|------------------------------------------------------|
| <b>High</b>   | Likely to cause service disruption   | Database changes, network changes, major upgrades  | Careful planning, maintenance window, full approvals |
| <b>Medium</b> | May cause limited impact             | Application updates, configuration changes         | Test, get approvals, monitor closely                 |
| <b>Low</b>    | Unlikely to cause significant impact | Non-critical config updates, documentation changes | Can be done with standard process                    |

## 4. CHANGE AND APPROVAL PROCESS

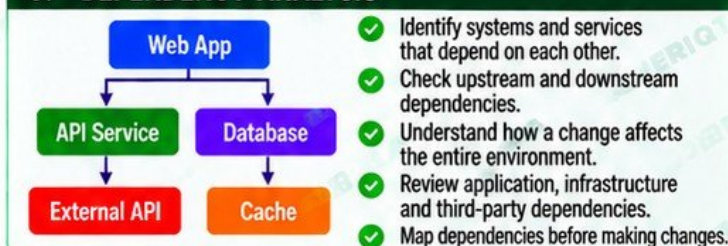


## 5. MAINTENANCE WINDOWS



- ✓ Scheduled time to make changes safely.
- ✓ Inform all stakeholders before the window.
- ✓ Confirm affected systems and services.
- ✓ Ensure rollback plan is ready.
- ✓ Keep the window as short as possible.
- ✓ Monitor closely during and after changes.

## 6. DEPENDENCY ANALYSIS



## 7. REMEDIATION PLANNING



- ✓ Define the problem and desired outcome.
- ✓ Identify possible remediation options.
- ✓ Evaluate risk, effort and impact for each option.
- ✓ Choose the best option and create an execution plan.
- ✓ Include rollback plan in case of issues.
- ✓ Assign owners and set target dates.
- ✓ Track progress and validate the fix.

## 8. QUESTIONS TO ASK

- ? What is the business impact if we make this change?
- ? What is the risk if we do nothing?
- ? Are there dependencies we might affect?
- ? Do we need a maintenance window?
- ? What approvals are required?
- ? How will we test the change?
- ? What is the rollback plan?
- ? Who needs to be informed?
- ? Can this be combined with other changes?



## 9. BEST PRACTICES

- ✓ Use a risk-based approach.
- ✓ Document all decisions.
- ✓ Get the right approvals.
- ✓ Test changes in a non-production environment when possible.
- ✓ Communicate with all stakeholders.
- ✓ Have a clear rollback plan.
- ✓ Monitor after the change.
- ✓ Review and learn from each change.

## 10. JUNIOR ENGINEER TIP



- ✓ Always understand the impact before making a change.
- ✓ Ask if you are not sure.
- ✓ Never make changes without approval.
- ✓ Follow the defined change process.
- ✓ Document what you do and why.
- ✓ Learn from past changes and incidents.
- ✓ Start with small, low risk changes and build experience.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Stabilizing Production Without Introducing New Failures

Establish baselines, validate monitoring, document safe changes, and verify improvements.

## 1. WHAT STABILIZING PRODUCTION MEANS

-  Keep services reliable, available and performing as expected.
-  Reduce risk while making controlled and safe improvements.
-  Understand current state before making any changes.
-  Validate that monitoring and alerts are working correctly.
-  Make improvements without introducing new failures.

## 2. STABILIZATION WORKFLOW



## 3. ESTABLISH BASELINES

-  Collect current performance metrics (CPU, memory, disk, network, latency).
-  Identify normal traffic patterns and peak usage times.
-  Review application and system logs for errors and warnings.
-  Document current configuration and versions.
-  Use baselines to compare results after changes.

## 4. VALIDATE MONITORING AND ALERTS

-  Confirm key metrics are being collected and are accurate.
-  Verify alert thresholds and notification channels.
-  Test alerting by simulating issues where possible.
-  Check dashboards for completeness and correctness.
-  Identify gaps in monitoring and add missing coverage.

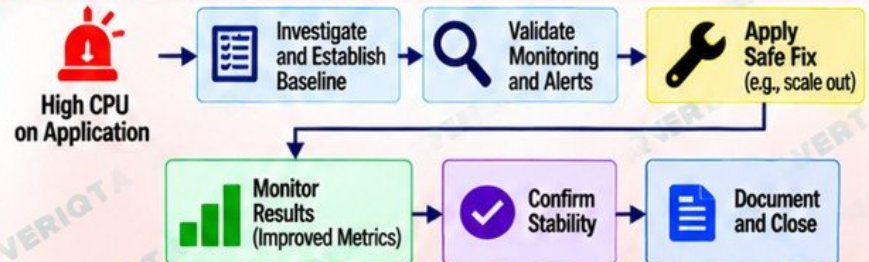
## 5. PLAN AND MAKE SAFE CHANGES

-  Understand the change and expected outcome.
-  Check dependencies and potential impact on other systems.
-  Get required approvals and follow change management process.
-  Make small, incremental changes where possible.
-  Have a rollback plan ready.
-  Implement the change during an approved maintenance window.

## 6. VERIFY IMPROVEMENTS

-  Compare metrics before and after the change.
-  Check application logs for new errors.
-  Confirm alerts are still working correctly.
-  Validate with users and stakeholders that the issue is resolved.
-  Document the results and close the change.

## 7. EXAMPLE STABILIZATION SCENARIO



## 8. COMMON RISKS

- ✗ Making changes without understanding dependencies.
- ✗ Disabling alerts or monitoring during changes.
- ✗ Applying untested fixes in production.
- ✗ Ignoring early warning signs after a change.
- ✗ Not having a rollback plan.
- ✗ Assuming the issue is fixed without verifying.

## 9. BEST PRACTICES

- ✓ Always establish baselines first.
- ✓ Validate monitoring and alerts regularly.
- ✓ Use change management and get approvals.
- ✓ Make small, incremental changes.
- ✓ Have and test a rollback plan.
- ✓ Monitor closely after changes.
- ✓ Document everything.
- ✓ Learn from each incident and improvement.

## 10. JUNIOR ENGINEER TIPS

- 💡 Take time to understand the system before making changes.
- ✓ Always check monitoring and logs.
- ✓ Ask questions if you are unsure.
- ✓ Start with low risk changes.
- ✓ Follow the approved process.
- ✓ Verify results and document what you did.
- ✓ Be patient and avoid rushing changes.
- ✓ Learn from each experience.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

# Creating the Missing Production Documentation

Build infrastructure inventories, dependency maps, service records, access matrices, and operational runbooks.

## 1. WHY PRODUCTION DOCUMENTATION MATTERS



Reduces operational risk.



Helps teams respond faster to incidents.



Improves knowledge sharing and handover.



Prevents mistakes and misconfigurations.



Supports compliance and audits.



Ensures business continuity when key people leave.

## 2. KEY DOCUMENTATION TYPES

### Infrastructure Inventory



List all resources (servers, VMs, containers, databases, networks, etc.)

### Dependency Maps



Show how services, applications and infrastructure depend on each other.

### Service Records



Document purpose, owner, technology, endpoints and critical information for each service.

### Access Matrices



Track who has access to what (systems, accounts, environments)

### Operational Runbooks



Step-by-step procedures for common operational tasks and incident response.

### Architecture Diagrams

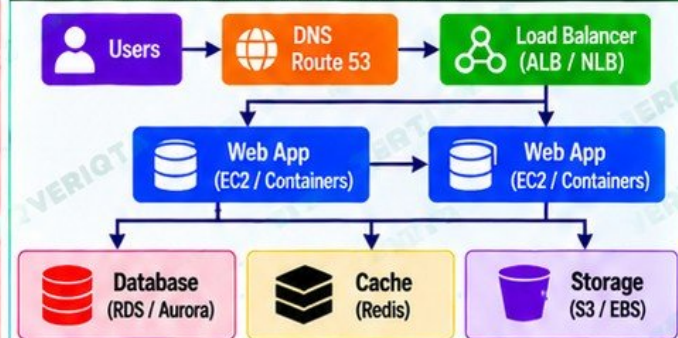


Visualize environments, components and data flow.

## 3. EXAMPLE INFRASTRUCTURE INVENTORY

| Resource Type | Example                     | Key Details to Document                       |
|---------------|-----------------------------|-----------------------------------------------|
| Compute       | EC2 / VM / Kubernetes node  | Name, region, OS, size, purpose, owner        |
| Database      | RDS / Aurora / PostgreSQL   | Version, endpoint, size, backup, owner        |
| Networking    | VPC / Subnet / Firewall     | CIDR, routing, security rules, owner          |
| Application   | Web app / API service       | Purpose, environment, endpoints, dependencies |
| Storage       | S3 / EBS / NFS              | Size, retention, backup, owner.               |
| Third-Party   | Payment API / Auth provider | Service name, purpose, contact, SLA           |

## 4. EXAMPLE DEPENDENCY MAP



## 5. EXAMPLE ACCESS MATRIX

| System / Resource  | Read Access   | Write Access  | Admin Access   | Owner         |
|--------------------|---------------|---------------|----------------|---------------|
| Production VPC     | Network Team  | Platform Team | Cloud Admins   | Infra Team    |
| RDS Database       | App Team      | DB Team       | DB Admins      | Data Team     |
| Kubernetes Cluster | Dev Team      | Platform Team | SRE Team       | Platform Team |
| Production Servers | Ops Team      | SRE Team      | Infrastructure | Infra Team    |
| S3 Bucket (Logs)   | Security Team | Platform Team | Cloud Admins   | Security Team |

## 6. KEY RUNBOOK SECTIONS (EXAMPLE)

| Runbook Title and Purpose | Example: Restart a Service         |
|---------------------------|------------------------------------|
| Prerequisites             | 1 Check current service status     |
| Step-by-Step Procedure    | 2 Review recent logs               |
| Verification Steps        | 3 Safely restart the service       |
| Troubleshooting Guide     | 4 Verify the service is running    |
| Rollback Procedure        | 5 Check metrics and error logs     |
| Contacts and Escalation   | 6 If issues, follow rollback steps |
| Last Reviewed and Version | 7 Notify stakeholders              |
|                           | 8 Document the action and outcome  |

## 7. COMMON GAPS IN DOCUMENTATION

- ✗ Missing or outdated infrastructure inventory.
- ✗ No dependency maps.
- ✗ No clear service ownership.
- ✗ Incomplete access records.
- ✗ Runbooks don't exist or are outdated.
- ✗ Diagrams are missing or unclear.
- ✗ Critical information only in people's heads.

## 8. BEST PRACTICES

- ✓ Keep documentation simple and up to date.
- ✓ Use clear and consistent templates.
- ✓ Include diagrams for complex systems.
- ✓ Assign owners and review regularly.
- ✓ Store documentation in a central location (e.g., Git repository or internal wiki).
- ✓ Link runbooks to monitoring and alerting.
- ✓ Validate documentation through real use.

## 9. JUNIOR ENGINEER TIPS



- ✓ Start with what you can find.
- ✓ Talk to teammates and ask questions.
- ✓ Document as you learn.
- ✓ Use diagrams to understand the environment.
- ✓ Keep records of access and important details.
- ✓ Improve documentation gradually.
- ✓ Review and update runbooks after incidents.
- ✓ Be proactive and help close documentation gaps.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

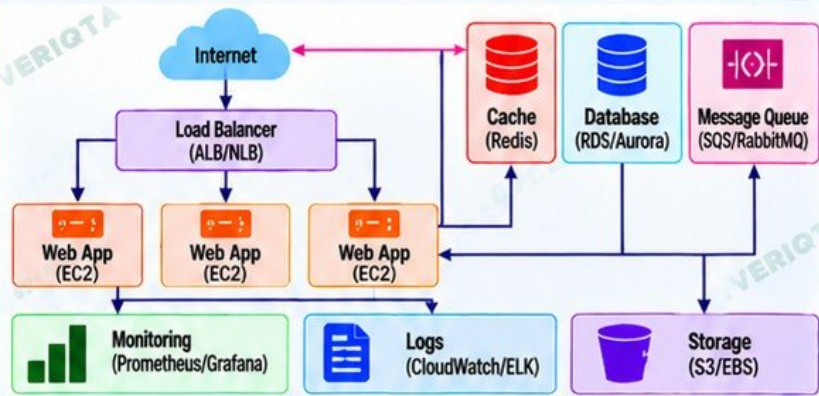
# Capstone: Take Ownership of an Undocumented Production Environment

Work through a simulated production handover, from initial discovery to an evidence-based stabilization and ownership report.

## 1. SCENARIO OVERVIEW

|                                                                                   |           |                                                                                        |
|-----------------------------------------------------------------------------------|-----------|----------------------------------------------------------------------------------------|
|  | Company   | E-commerce platform (undocumented environment).                                        |
|  | Services  | Web app, API, database, cache, message queue, monitoring (partially configured).       |
|  | Your Role | You are the new production engineer taking over the environment.                       |
|  | Goal      | Discover, understand, stabilize and produce a complete ownership report with evidence. |

## 2. ENVIRONMENT ARCHITECTURE (DISCOVER AND MAP)



## 3. CAPSTONE TASK FLOW

|    |                                                                                     |                     |                                                                            |
|----|-------------------------------------------------------------------------------------|---------------------|----------------------------------------------------------------------------|
| 01 |  | Initial Discovery   | Explore the environment, list all resources, and identify what is missing. |
| 02 |  | Document and Map    | Create infrastructure inventory, dependency map and service records.       |
| 03 |  | Analyze Risks       | Find single points of failure, security gaps and operational risks.        |
| 04 |  | Validate Monitoring | Check dashboards, metrics, logs and alerts. Identify blind spots.          |
| 05 |  | Stabilize           | Fix critical and low risk issues within approved scope.                    |
| 06 |  | Hand over Ownership | Produce an evidence-based stabilization and ownership report.              |

## 4. KEY DISCOVERY CHECKLIST

- ✓ List all infrastructure resources.
- ✓ Identify applications and services.
- ✓ Map dependencies between components.
- ✓ Check access and ownership information.
- ✓ Review monitoring, logging and alerting.
- ✓ Look for configuration and security issues.
- ✓ Identify undocumented components.
- ✓ Understand data flows and third-party services.

## 5. EXAMPLE FINDINGS

-  **Missing Documentation**  
No up-to-date architecture diagram.
-  **Configuration Issues**  
Hardcoded credentials in a config file.
-  **Single Point of Failure**  
Only one EC2 instance for web app.
-  **Monitoring Gap**  
No alert for high database CPU usage.
-  **Security Risk**  
Overly permissive IAM role.
-  **Operational Risk**  
No documented maintenance window.

## 6. STABILIZATION ACTIONS (EXAMPLES)

| Issue             | Action Taken                                         | Risk Level | Result                      |
|-------------------|------------------------------------------------------|------------|-----------------------------|
| High CPU on DB    | Optimized queries and increased instance size        | High       | CPU reduced from 95% to 30% |
| Missing alerts    | Created CloudWatch alerts for critical metrics       | High       | Alerting now active         |
| Outdated OS       | Scheduled patching during maintenance window         | Medium     | System updated              |
| No documentation  | Created runbooks, architecture diagram and inventory | Medium     | Improved visibility         |
| Single web server | Deployed additional instance behind load balancer    | High       | Improved availability       |

## 7. OWNERSHIP REPORT OUTLINE

|                                                                                     |                         |                                                               |
|-------------------------------------------------------------------------------------|-------------------------|---------------------------------------------------------------|
|  | Executive Summary       | Current state, key findings and overall stability assessment. |
|  | Environment Inventory   | List of all resources and configurations.                     |
|  | Architecture Diagram    | Updated diagram with dependencies.                            |
|  | Key Risks and Issues    | Identified risks and their impact.                            |
|  | Changes Implemented     | What was fixed and why.                                       |
|  | Monitoring and Alerting | Current status and improvements.                              |
|  | Remaining Work          | Known issues and recommendations.                             |
|  | Handover Notes          | Key contacts, access details and next steps.                  |

## 8. SUCCESS CRITERIA

- ✓ Complete and accurate infrastructure inventory.
- ✓ Clear and correct architecture diagram.
- ✓ Identified and documented key risks.
- ✓ Validated monitoring and alerting.
- ✓ Implemented safe and approved changes.
- ✓ Produced a detailed ownership report with evidence.

## 9. COMMON CHALLENGES

- ✗ Incomplete or missing information.
- ✗ Hidden dependencies.
- ✗ Legacy or outdated systems.
- ✗ Access and permission issues.
- ✗ Unexpected errors during fixes.
- ✗ Lack of documentation and runbooks.

## 10. JUNIOR ENGINEER TIPS

-  Take it step by step and document everything.
- ✓ Ask questions when something is unclear.
- ✓ Focus on low risk changes first.
- ✓ Always validate before and after a change.
- ✓ Keep evidence (screenshots, logs, outputs).
- ✓ Be honest about what you don't know.
- ✓ Communicate progress with stakeholders.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta